AGENCY OS

COMPLETE CURRENT SYSTEM AUDIT & IMPLEMENTATION STATUS REPORT

Read-only application, database, workflow, security, performance, UX, and architecture assessment

AUDIT SNAPSHOT

AUDIT DATE	August 12, 2026
REPOSITORY COMMIT	e14e68ac98fa7dc6f43eb6282b6bcde80a9e4912
BRANCH	arena/019ff5ec-website
SCOPE	102 tracked files / 26 pages / 13 migrations / 24 tables
MODE	Audit only - no application code or database changes

BOTTOM LINE

Strong Supabase-backed foundation, but critical profile, password, account lifecycle, permission, public-form abuse, schema consistency, and deployment issues remain.

Prepared from repository evidence and available deployment metadata.

The repository remained clean throughout the audit.

1. Executive Summary

The application has a substantial implemented foundation:

- Closed Supabase authentication
- Metadata-driven roles and permissions
- Database Row Level Security (RLS)
- Public client landing page
- Dynamic form builder and anonymous submission flow
- Public database-backed portfolio
- CRM client, project, task, file, team, and notification systems
- Responsive staff shell
- Database security regression test suite

However, the product is not ready to be considered complete. The most important findings are:

1. Profile editing is broken. The UI calls update_own_enhanced_profile, but that RPC does not exist in any migration or in schema.sql.
2. Temporary-password enforcement is broken. New accounts are not marked must_change_password = true; the redirect can loop; enforcement is client-side; and mark_password_changed can clear the flag for an arbitrary user.
3. schema.sql and migrations are inconsistent. schema.sql omits migration 20260816000000_force_password_change_and_profile_fields.sql.
4. Account editing/deletion is incomplete. Changing a team member's email updates only profiles.email, not Supabase Auth. Deleting a member deletes only the profile, leaving the Auth identity behind.
5. Granular form and portfolio permissions are not independently usable through the UI. /admin/** requires admin.manage, even when a custom role has form.manage or portfolio.manage.
6. Employees receive submission.view by default. This permits direct database access to all intake/dynamic-form submissions, answers, attachments, and client PII.
7. The public form endpoint has no abuse controls: no rate limit, CAPTCHA, answer-size limits, or storage limits. Published forms can be spammed and can auto-create CRM clients/projects.
8. The legacy Intake backend remains active. Its tables, RPC, storage bucket, portal workflow, and notification trigger still exist.
9. The intended /intake -> /forms redirect is broken for unauthenticated visitors because /intake is not classified as a public route by AppShell.
10. The public deployment could not be validated. The production deployment URL discovered through GitHub redirected to Vercel authentication, while the repository homepage URL returned 404.

The strongest implemented end-to-end workflow is:

Public landing -> published dynamic form -> anonymous submission -> database answer snapshots -> CRM client match/create -> optional project -> admin notification -> admin response review.

It still needs abuse protection, deployment verification, clearer ownership/assignment after submission, and legacy Intake retirement.

2. Audit Scope and Verification Limits

Inspected

- 102 tracked files
- 61 TypeScript/TSX files
- 26 application page routes
- 1 API route
- 13 Supabase migrations
- 24 database tables
- 5 storage buckets
- supabase/schema.sql
- RLS policies, RPCs, triggers, constraints, indexes, and foreign keys
- Authentication context and route shell
- All major pages and component workflows
- Database regression test source
- GitHub and Vercel deployment status

Runtime limitations

- There is no .env.local in the checkout.
- There is no supabase/config.toml or linked-project metadata.
- Supabase URL, anon key, service-role key, Auth provider settings, and live migration state could not be inspected.
- node_modules is absent. The audit instruction prohibited installing packages, so local build, lint, and database tests were not run.
- Vercel reports a successful build for the audited commit, but there is no GitHub test check.
- The available Vercel deployment redirected to Vercel login, preventing public workflow smoke testing.

Database-dependent statuses therefore describe the repository implementation and explicitly identify where deployed state remains unverified.

3. Complete Feature Inventory and Status

Login

COMPLETE

WHAT EXISTS
Email/password login through Supabase; no signup mode; redirects internal users to /dashboard

WHAT'S MISSING / BROKEN
Live Auth configuration not inspectable

PRIORITY
High

Logout

COMPLETE

WHAT EXISTS
Supabase sign-out from top bar and client portal

WHAT'S MISSING / BROKEN
Errors are minimally surfaced

PRIORITY
Medium

Password reset

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Reset email through Supabase Auth; update-password mode at /auth

WHAT'S MISSING / BROKEN
Redirect configuration unverified; reset does not clear forced-password flag

PRIORITY
High

Public signup prevention

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
No signup UI; Auth trigger rejects permanent users without trusted service-role metadata

WHAT'S MISSING / BROKEN
Supabase dashboard signup setting cannot be verified

PRIORITY
Critical

Anonymous visitors

COMPLETE

WHAT EXISTS
Anonymous Auth supported for uploads; submit RPC is also granted to anon

WHAT'S MISSING / BROKEN
Anonymous provider setting cannot be verified

PRIORITY
High

Temporary passwords

PARTIALLY IMPLEMENTED

WHAT EXISTS
Admin manually enters an initial password; it is shown again in a one-time credentials modal

WHAT'S MISSING / BROKEN
Password is not generated; account is not marked for forced change

PRIORITY
High

Forced password change

BROKEN

WHAT EXISTS
must_change_password field and Profile UI exist

WHAT'S MISSING / BROKEN
Never set true; redirect can loop; backend remains accessible; arbitrary flag-clearing RPC

PRIORITY
Critical

User profile display

PARTIALLY IMPLEMENTED

WHAT EXISTS
Profile page, team cards, detailed team profile

WHAT'S MISSING / BROKEN
Enhanced fields depend on a migration missing from schema.sql

PRIORITY
High

User profile editing

BROKEN

WHAT EXISTS
UI exposes personal, professional, portfolio, social, and avatar fields

WHAT'S MISSING / BROKEN
Called RPC does not exist; avatar can upload before save fails

PRIORITY
Critical

User password change

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Authenticated user can call updateUser({password})

WHAT'S MISSING / BROKEN
Current password is collected but never verified

PRIORITY
High

Team directory

COMPLETE

WHAT EXISTS
Active staff directory, search, role/department filters, member profiles, client exclusion

WHAT'S MISSING / BROKEN
Browser behavior not smoke-tested

PRIORITY
Medium

Team management

PARTIALLY IMPLEMENTED

WHAT EXISTS
Create, view, edit, activate/deactivate, delete profile, assign role/job role

WHAT'S MISSING / BROKEN
Auth email synchronization and Auth-user deletion are missing

PRIORITY
Critical

Employee account creation

PARTIALLY IMPLEMENTED

WHAT EXISTS
Admin API creates a real Auth user and linked profile

WHAT'S MISSING / BROKEN
No generated password; forced change broken

PRIORITY
Critical

Manager account creation

PARTIALLY IMPLEMENTED

WHAT EXISTS
Manager system role can be selected during provisioning

WHAT'S MISSING / BROKEN
Same lifecycle defects as employee accounts

PRIORITY
Critical

Additional Admin creation

PARTIALLY IMPLEMENTED

WHAT EXISTS
Admin role can be selected during provisioning

WHAT'S MISSING / BROKEN
Same lifecycle defects; lockout protections incomplete

PRIORITY
Critical

Custom internal account creation

PARTIALLY IMPLEMENTED

WHAT EXISTS
Any active non-client custom role can be selected

WHAT'S MISSING / BROKEN
UI route still requires admin.manage

PRIORITY
High

Client login accounts

LEGACY / UNUSED

WHAT EXISTS
Existing client-role profiles can use /portal and link to CRM clients

WHAT'S MISSING / BROKEN
No current creation UI; portal reads only legacy Intake

PRIORITY
Medium

Job roles

COMPLETE

WHAT EXISTS
Database-driven employee job-role catalog; create/edit/disable/delete/assign

WHAT'S MISSING / BROKEN
Separate access-role and job-role concepts may confuse admins

PRIORITY
Medium

System roles

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Admin, Manager, Employee, Client seeded

WHAT'S MISSING / BROKEN
Admin role can be disabled or stripped of admin.manage

PRIORITY
Critical

Custom roles

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Create/edit/deactivate/delete and assign roles

WHAT'S MISSING / BROKEN
Some permission combinations have no reachable UI

PRIORITY
High

Permission catalog

PARTIALLY IMPLEMENTED

WHAT EXISTS
Metadata-driven permission table and backend add_permission RPC

WHAT'S MISSING / BROKEN
No UI to add/edit permission definitions

PRIORITY
Medium

Permission checkboxes

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Database-loaded grouped checkbox editor; exact grants saved

WHAT'S MISSING / BROKEN
No backend invariant preserving at least one administrator

PRIORITY
Critical

Client-side route protection

PARTIALLY IMPLEMENTED

WHAT EXISTS
AppShell checks required permission per route

WHAT'S MISSING / BROKEN
No middleware/server-render guard; /forms bypasses its declared permission

PRIORITY
High

Server API protection

COMPLETE

WHAT EXISTS
Team provisioning verifies JWT and admin.manage before service-role use

WHAT'S MISSING / BROKEN
Other writes rely on RLS rather than server APIs

PRIORITY
High

Supabase RLS

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
RLS enabled on all 24 application tables

WHAT'S MISSING / BROKEN
Default employee submission visibility is too broad; live state unverified

PRIORITY
Critical

Public landing page

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Responsive landing, forms, portfolio, login CTAs, dynamic records

WHAT'S MISSING / BROKEN
Deployment inaccessible for verification; marketing content hardcoded

PRIORITY
Critical

Public forms listing

PARTIALLY IMPLEMENTED

WHAT EXISTS
/forms loads RLS-visible published templates

WHAT'S MISSING / BROKEN
Also used as internal sidebar Forms destination; public counts misleading

PRIORITY
High

Dynamic form builder

PARTIALLY IMPLEMENTED

WHAT EXISTS
Create forms; 10 question types; options; required/help/placeholder/mapping; reorder; preview

WHAT'S MISSING / BROKEN
Empty/invalid forms can be published; granular manager UI blocked

PRIORITY
High

Form sections

PARTIALLY IMPLEMENTED

WHAT EXISTS
Questions grouped under headings; progress displayed

WHAT'S MISSING / BROKEN
Not a multi-step wizard; all sections remain on one page

PRIORITY
Medium

Conditional form logic

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Exact-match and multi-choice containment; frontend/RPC skip hidden fields

WHAT'S MISSING / BROKEN
Free-text conditions; no dependency-cycle validation

PRIORITY
Medium

Form publishing

COMPLETE

WHAT EXISTS
Draft/published/disabled/archived lifecycle; RLS exposes only published forms

WHAT'S MISSING / BROKEN
No publish-readiness validation

PRIORITY
High

Form editing/versioning

COMPLETE

WHAT EXISTS
Question edits bump version; responses retain snapshots

WHAT'S MISSING / BROKEN
Template detail edits do not bump version

PRIORITY
Medium

Anonymous form submission

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Public RPC validates key field types and stores responses

WHAT'S MISSING / BROKEN
No anti-spam/rate limit; text/date lengths and types weakly validated

PRIORITY
Critical

Submission answer storage

COMPLETE

WHAT EXISTS
Separate submission and answer rows; frozen question snapshot

WHAT'S MISSING / BROKEN
Visible unanswered optional questions can store null answer rows

PRIORITY
Medium

Form file uploads

BROKEN / UNSAFE

WHAT EXISTS
Private bucket, owner folders, attachment metadata, signed downloads

WHAT'S MISSING / BROKEN
No size/type limits; removed/abandoned uploads never deleted

PRIORITY
Critical

Form submission review

PARTIALLY IMPLEMENTED

WHAT EXISTS
Per-form Responses tab; answers; downloads; archive/restore

WHAT'S MISSING / BROKEN
No global inbox, assignment, review status, export, or pagination

PRIORITY
High

Form submission notifications

COMPLETE

WHAT EXISTS
Trigger notifies active admins and links to exact response

WHAT'S MISSING / BROKEN
Requires deployed migration; no email

PRIORITY
High

Legacy Intake

LEGACY / UNUSED

WHAT EXISTS
Tables, RPC, storage, portal reads, automations, notification trigger

WHAT'S MISSING / BROKEN
Competes with dynamic-form backend; public redirect broken

PRIORITY
High

Public portfolio

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Public RPC, published-only records, signed images, categories, detail pages

WHAT'S MISSING / BROKEN
N+1 signed URLs; deployment unverified

PRIORITY
High

Portfolio management

PARTIALLY IMPLEMENTED

WHAT EXISTS
Create/edit/delete/archive; categories; images; cover; feature; publish; project order

WHAT'S MISSING / BROKEN
Gallery images cannot be reordered; independent permission UI blocked

PRIORITY
Medium

CRM clients

COMPLETE

WHAT EXISTS
CRUD, details, search, project relation, RLS

WHAT'S MISSING / BROKEN
Email is not unique; concurrent public submissions can duplicate clients

PRIORITY
High

Client public functionality

PARTIALLY IMPLEMENTED

WHAT EXISTS
Public form submission and portfolio require no login

WHAT'S MISSING / BROKEN
No tracking, confirmation email, or client-facing status

PRIORITY
High

Legacy client portal

LEGACY / UNUSED

WHAT EXISTS
/portal lists linked legacy Intake submissions

WHAT'S MISSING / BROKEN
Does not show dynamic submissions, projects, tasks, or files

PRIORITY
Medium

Project CRUD

PARTIALLY IMPLEMENTED

WHAT EXISTS
Create/edit/delete, client, type, status, dates, budget, progress, phase

WHAT'S MISSING / BROKEN
No owner, manager, priority, or client-account access

PRIORITY
High

Project assignment

PARTIALLY IMPLEMENTED

WHAT EXISTS
project_members; active employees assignable; RLS controls reads

WHAT'S MISSING / BROKEN
UI excludes managers/admins; auto-created projects remain unassigned

PRIORITY
High

Project notifications

COMPLETE

WHAT EXISTS
Assignment and progress/status/phase/due-date triggers

WHAT'S MISSING / BROKEN
No unassignment notification or preferences

PRIORITY
Medium

Task CRUD

PARTIALLY IMPLEMENTED

WHAT EXISTS
Create, status board, priority, assignee, due date, delete

WHAT'S MISSING / BROKEN
General task page can assign only current user; permission controls inconsistent

PRIORITY
High

Task assignment integrity

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
FK to profiles and assignment trigger

WHAT'S MISSING / BROKEN
Assignee need not be active, internal, or a project member

PRIORITY
High

My Tasks

NOT IMPLEMENTED

WHAT EXISTS
None

WHAT'S MISSING / BROKEN
No current-user filter or dedicated employee task view

PRIORITY
High

Overdue tasks

NOT IMPLEMENTED

WHAT EXISTS
Due dates are stored

WHAT'S MISSING / BROKEN
No overdue calculation, filter, metric, reminder, or escalation

PRIORITY
High

Task notifications

PARTIALLY IMPLEMENTED

WHAT EXISTS
Notification on initial assignment/reassignment

WHAT'S MISSING / BROKEN
No status, due-soon, overdue, or completion notifications

PRIORITY
Medium

Employee workspace

PARTIALLY IMPLEMENTED

WHAT EXISTS
Shared dashboard with assigned projects/clients/tasks/files under RLS

WHAT'S MISSING / BROKEN
No My Tasks; all project tasks shown; Forms leaves workspace

PRIORITY
High

Manager workspace

PARTIALLY IMPLEMENTED

WHAT EXISTS
Shared shell with all-project/client/task/file permissions

WHAT'S MISSING / BROKEN
No manager-specific dashboard or triage queue

PRIORITY
Medium

Admin workspace

PARTIALLY IMPLEMENTED

WHAT EXISTS
Team, roles, forms, portfolio, clients, projects, assignments

WHAT'S MISSING / BROKEN
Monolithic tabs and lifecycle inconsistencies

PRIORITY
High

Project file upload/download

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Private bucket, project paths, signed downloads, RLS

WHAT'S MISSING / BROKEN
No limits; delete policy mismatch can leave broken records

PRIORITY
High

Storage buckets

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Five buckets with RLS/storage policies

WHAT'S MISSING / BROKEN
No size/MIME restrictions; avatar bucket is public

PRIORITY
High

In-app notifications

PARTIALLY IMPLEMENTED

WHAT EXISTS
Persistent inbox/dropdown, unread state, filters, links, delete

WHAT'S MISSING / BROKEN
Count based on only 20 rows; no realtime; page capped at 100

PRIORITY
Medium

Email notifications

NOT IMPLEMENTED

WHAT EXISTS
Supabase Auth password-reset email only

WHAT'S MISSING / BROKEN
No business email sender

PRIORITY
High

Search

PARTIALLY IMPLEMENTED

WHAT EXISTS
Project, client, team, notification search

WHAT'S MISSING / BROKEN
Top bar is project-only; no global search; client-side filtering

PRIORITY
Medium

Filters

PARTIALLY IMPLEMENTED

WHAT EXISTS
Project status, team, notification, portfolio category filters

WHAT'S MISSING / BROKEN
No task overdue/assignee or submission filters

PRIORITY
Medium

Dashboard

PARTIALLY IMPLEMENTED

WHAT EXISTS
Live accessible project/task/client/notification metrics

WHAT'S MISSING / BROKEN
Loads complete datasets; no role-specific queues

PRIORITY
Medium

Reports

PARTIALLY IMPLEMENTED

WHAT EXISTS
Average progress, task counts, project health

WHAT'S MISSING / BROKEN
No date range, financial reports, export, or pagination

PRIORITY
Medium

Activity logs

NOT IMPLEMENTED

WHAT EXISTS
Interactions/comments tables exist

WHAT'S MISSING / BROKEN
No audit/event log or activity feed

PRIORITY
High

Responsive/mobile

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Responsive grids, public menus, staff bottom navigation

WHAT'S MISSING / BROKEN
Not browser-tested; mobile nav can contain eight tiny items

PRIORITY
Medium

Error handling

PARTIALLY IMPLEMENTED

WHAT EXISTS
Most pages show inline database errors

WHAT'S MISSING / BROKEN
No global error boundary; some errors ignored

PRIORITY
Medium

Loading states

COMPLETE

WHAT EXISTS
Major data pages show loading indicators

WHAT'S MISSING / BROKEN
No skeletons or server-rendered public data

PRIORITY
Low

Empty states

COMPLETE

WHAT EXISTS
Purposeful empty states across major screens

WHAT'S MISSING / BROKEN
Some next actions could be clearer

PRIORITY
Low

Settings

NOT IMPLEMENTED

WHAT EXISTS
Redirect-style page linking to Profile

WHAT'S MISSING / BROKEN
No workspace/system settings editor

PRIORITY
Medium

Templates

NOT IMPLEMENTED

WHAT EXISTS
Placeholder route

WHAT'S MISSING / BROKEN
No template functionality

PRIORITY
Low

AI Assistant

NOT IMPLEMENTED

WHAT EXISTS
Placeholder route

WHAT'S MISSING / BROKEN
No provider or assistant workflow

PRIORITY
Low

Theme/accent controls

COMPLETE

WHAT EXISTS
Persisted light/dark theme and accent context

WHAT'S MISSING / BROKEN
Minor maintainability concerns only

PRIORITY
Low

Interactions/comments

LEGACY / UNUSED

WHAT EXISTS
Tables and RLS exist

WHAT'S MISSING / BROKEN
No frontend workflows use them

PRIORITY
Low

Database regression tests

IMPLEMENTED BUT NEEDS REVIE

WHAT EXISTS
Extensive PGlite security test source

WHAT'S MISSING / BROKEN
Not run locally; no CI check; profile defects not covered

PRIORITY
High

4. Authentication and User Management Audit

Public visitors

Can a visitor create an account?

Intended answer: No.

Evidence:

- /auth contains login and reset only.
- guard_internal_account_creation() rejects permanent Auth inserts or anonymous conversion without agency_os_admin_provisioned.
- The service-role API is the only browser-accessible provisioning path.
- Anonymous users are deliberately exempted from profile creation.

Caveat: Supabase's dashboard-level Allow New Users setting could not be verified. The database trigger should still reject signup if the latest migration is applied.

Can a client submit without login?

Yes.

- /forms and /f/[slug] are public.
- The page attempts anonymous sign-in.
- submit_dynamic_form is executable by anon and authenticated.
- Anonymous submission creates no internal profile.
- A CRM client may be matched or created by mapped email.
- Optional project creation is controlled by form settings.
- File upload requires a usable anonymous Auth session.

Can a client accidentally become an internal user?

Not through the intended repository path.

- Anonymous Auth users do not receive profiles.
- Permanent public signup is blocked by an Auth-table trigger.
- Form submission creates CRM records, not staff profiles.
- The provisioning API excludes the Client role.

Internal account creation

First Admin	One-time `scripts/bootstrap-adm
Employee	Admin Team Management -> prote
Manager	Same flow, with Manager role se
Additional Admin	Same flow, with Admin role sele

Custom internal role	**Same flow, with active custom r**
Client login	**No current creation UI; existin**

Temporary password behavior

- The Admin manually chooses the password.
- It is sent directly to Supabase Auth Admin API.
- It is displayed again after successful creation.
- It is not stored in profiles.
- The credentials modal can copy the email only, not the password or combined credentials.

Forced password change

This workflow is broken:

1. Provisioning does not write must_change_password = true.
2. The column defaults to false.
3. The Auth context redirects using window.location.href = '/profile'.
4. If the flag is true while already on /profile, that can cause a reload loop.
5. The database does not block projects, tasks, or other operations while the flag is true.
6. mark_password_changed(p_user_id) accepts any user ID and does not check p_user_id = auth.uid().
7. Password reset uses the basic password update function and does not clear the flag.

Account editing and deletion defects

Email changes

admin_update_team_member updates profiles.email, but does not update auth.users.email.

Result:

- Team UI can display a new email.
- Login may still require the old Auth email.
- Auth and application identity become inconsistent.

Account deletion

admin_delete_team_member deletes from profiles only.

Result:

- The Supabase Auth user remains.
- Login credentials are not deleted.
- The user has no usable profile/permissions but still exists as an Auth identity.
- Delete Team Member is not true account deletion.

There is no Admin password-reset or resend-credentials workflow.

5. Profile Field Control Audit

Intended user-editable fields shown in /profile

- Full name
- Profile photo
- Phone
- WhatsApp
- Location
- Bio
- Job title
- Skills
- Experience
- Certifications
- Previous projects
- Portfolio URL
- LinkedIn
- Behance
- Instagram
- Facebook
- X/Twitter
- Personal website
- Custom social links
- Password

Actual result

Except for password update and raw avatar upload, profile saving is broken because the page calls a nonexistent RPC.

The older update_own_profile RPC does exist and safely updates only:

- Full name
- Avatar URL
- Agency name
- Agency website
- Phone
- Bio

The current Profile page does not use that RPC.

Admin-controlled fields

- Auth/application email, intended but currently only profile email changes
- Access role
- Account status
- Employee job role
- Department
- Specialization
- Client linkage for legacy client accounts
- Initial password during creation

Mixed or unclear control

- job_title is editable in both Admin Team Management and the user Profile UI.
- department and specialization are Admin-editable but not user-editable.
- Birthday, social media, and notes are intended as self-editable but cannot currently be saved.

6. Roles and Permissions Audit

Architecture

The database uses:

- app_roles: system and custom access roles
- permissions: capability catalog
- role_permissions: many-to-many grants
- profiles.role_id: assigned access role
- profiles.role: legacy enum retained for compatibility
- has_permission(): central database authorization helper
- get_user_permissions(): effective permission list
- Permission-aware RLS and RPC checks

This is a strong architectural direction. Direct project/client/task/form/portfolio operations are generally protected by RLS or permission-checked RPCs rather than only by hidden buttons.

Important enforcement inconsistencies

1. /admin/** requires admin.manage.
2. form.manage alone cannot open the form builder.
3. portfolio.manage alone cannot open portfolio management.
4. Role-management permissions alone cannot reach their UI.
5. /forms is declared to require submission.view, but AppShell treats /forms as public before evaluating route permissions.
6. Employee submission.view grants backend access to all form submissions, answers, and attachments.
7. Defined permissions with no meaningful UI include submission.assign, settings.edit, file.edit, permission.manage, and Manager's employee.edit.
8. Admin permissions can be removed or the Admin role disabled without preserving a usable administrator.

Default capability matrix

Staff workspace

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If workspace.access

Dashboard

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If dashboard.view

View all projects

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
If project.view_all

View assigned projects

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If project.view plus membership

Create/edit/delete projects

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
Per permission

Assign project members

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
If project.assign

View all clients

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
If client.view_all

Create/edit clients

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
Per permission

Delete clients

Yes

MANAGER
No

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
If client.delete

View tasks in accessible projects

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If task.view

Create tasks

Yes

MANAGER
Yes

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
If task.create

Edit task status

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If permitted by policy

Upload project files

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If file.upload

View submissions

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
Own by owner RLS only

CUSTOM ROLE
If submission.view

Manage form definitions

Yes

MANAGER
No

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
DB permits form.manage; UI also requires admin.manage

Manage public portfolio

Yes

MANAGER
No

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
DB permits portfolio.manage; UI also requires admin.manage

View team directory

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If employee.view

Create team Auth accounts

Yes

MANAGER
No

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
Requires admin.manage

Manage roles/permissions

Yes

MANAGER
No

EMPLOYEE
No

CLIENT
No

CUSTOM ROLE
DB permissions plus UI admin requirement

Notifications

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If notification.view

Reports

Yes

MANAGER
Yes

EMPLOYEE
Yes

CLIENT
No

CUSTOM ROLE
If report.view

Client portal

No

MANAGER
No

EMPLOYEE
No

CLIENT
Yes

CUSTOM ROLE
Legacy Client role only

7. Forms Audit

Actual dynamic form flow

Admin with admin.manage and form.manage -> Administration / Forms -> Create template -> /admin/forms/[id] -> Add, edit, reorder questions -> Publish -> RLS exposes template and questions -> Visitor opens /forms -> Visitor opens /f/[slug] -> Anonymous session attempted -> submit_dynamic_form() -> form_submissions -> form_submission_answers with snapshots -> form_submission_attachments -> CRM client match/create by email -> Optional project creation -> Notification trigger -> Admin Responses tab.

Implemented question types

1. Short text
2. Long text
3. Single choice
4. Multiple choice
5. Yes/No
6. Dropdown
7. Number
8. Date
9. File upload
10. Rating

Implemented builder controls

- Label
- Help text
- Placeholder
- Required
- Answer options
- Rating maximum
- CRM mapping: name, email, phone, company
- Position/reordering
- Section heading
- Conditional visibility
- Preview
- Create project on submit
- Duplicate
- Publish, disable, archive, delete

Server validation present

- Published status
- Required questions
- Choice values belong to configured options
- Multiple-choice values belong to configured options
- Yes/no values
- Number format
- Rating range
- File answer must be an array

- Conditional hidden fields are skipped

Missing or unsafe validation

- No maximum number of answers.
- No maximum string length.
- Short, long, and date values are not strictly required to be strings.
- Date values are not validated as dates.
- Email mapping is not validated as an email.
- File metadata is trusted apart from the path prefix.
- File object existence is not checked.
- Publishing an empty form is allowed.
- Conditional values are not checked against trigger options.
- No conditional cycle detection.
- No rate limiting, CAPTCHA, honeypot, or duplicate-submission control.

Submission workflow gaps

- No reviewed/contacted/in-progress/rejected status.
- No submission assignment.
- No notes or collaboration.
- No global submission inbox.
- No pagination or export.
- No confirmation email.
- Auto-created projects have no manager or members.
- Public clients cannot track dynamic submissions through /portal.

Legacy Intake result

The old system is not removed. Still present:

- intake_forms
- intake_projects
- intake_attachments
- submit_intake_form
- intake-files bucket
- Intake RLS
- Intake client portal reads
- Intake notifications
- Intake helpers and TypeScript types

The UI page is a client-side redirect, but /intake is not public in AppShell. An unauthenticated visitor is sent to /auth before the redirect component can mount.

Current architecture still has two submission backends:

Dynamic forms -> form_ tables Legacy Intake -> intake_ tables.

Only the first has an active public form UI, but the second remains operational and is still used by the legacy Client portal.

8. Client Landing Page Audit

Source-level intended flow

/

- Request a New Project -> /forms -> published form -> /f/[slug] -> submit
- Portfolio -> /portfolio
- Sign in -> /auth

Verified in source:

- No signup CTA.
- Request CTAs point to /forms, not /intake.
- Published forms load dynamically.
- Unpublished forms are hidden by RLS.
- Portfolio routes are outside the staff shell.
- Login remains visible.
- Responsive desktop and mobile menus exist.

Problems

1. The internal sidebar also links Forms to public /forms.
2. Clicking it removes the staff shell and appears to leave the application.
3. Public form response counts are unreliable because RLS may return zero or only the visitor's own count.
4. The repository homepage URL returns 404.
5. The discoverable Vercel deployment redirects unauthenticated users to Vercel login.
6. No confirmed public production URL could be smoke-tested.

If Vercel protection applies to the real production alias, the entire public-client experience is operationally unavailable.

9. Portfolio Audit

Admin portfolio management

Implemented:

- Create and edit
- Hard delete
- Archive and restore
- Image upload
- Cover selection
- Categories and category enable/disable
- Featured flag
- Publish and unpublish
- Project ordering

Missing:

- Gallery image reordering
- Image alt-text editing
- Bulk upload progress and retry
- Transactional storage/database deletion
- Independent UI access for a role with only portfolio.manage

Public portfolio

Implemented:

- Public list and project details
- Published/non-archived filtering
- Category filters
- Images
- Responsive layout
- Database-backed narrow public RPCs
- Private bucket signed URLs
- Draft and archived image protection

Project records are not hardcoded. Marketing and About content are hardcoded.

10. Projects and Tasks Audit

Projects implemented

- Create, edit, delete
- Required CRM client relationship
- Type and status
- Start and due dates
- Budget and currency
- Phase and phase name
- Progress
- Project members
- Assigned-project RLS
- Assignment notifications
- Progress/status notifications

Projects missing

- Project owner
- Project manager
- Project priority
- Project-level team role
- Client portal visibility
- Activity timeline
- Comments UI
- Interaction UI
- Files inside project details
- Pagination
- Archive instead of hard delete
- Assignment during auto-created project flow

Tasks implemented

- Create and delete
- Assignee
- Status
- Priority
- Due date
- Board columns
- Project task view
- Assignment notification
- Highlight from notification query parameter

Tasks broken or incomplete

- General /tasks creation offers only the current user as assignee.
- Project detail shows New Task without checking task.create.
- General board shows controls where database policy may deny changes.
- No My Tasks.

- No overdue state.
- No due-soon reminders.
- No task filters or search.
- No task detail route.
- No comments or attachment workflow.
- Database does not require the assignee to be a project member.

11. Notifications Audit

Database structure

Notifications include recipient, actor, project, task, submission, type, title, message, action URL, metadata, read timestamp, and created timestamp. RLS restricts records to the recipient and requires notification.view.

Trigger coverage

Dynamic form submission to Admin	Yes
Legacy Intake submission to Admin	Yes
Project assignment to user	Yes
Project progress/status update to members	Yes
Task assignment/reassignment to assignee	Yes
Task status change	No
Task completion	No
Due soon or overdue	No
Project unassignment	No
File upload	No

Notification UI

Implemented:

- Dropdown
- Inbox page

- Polling every 20 seconds in the shell
- Unread count
- Mark read/unread
- Mark all read
- Delete
- Clear read
- Type filters
- Search
- Action links

Problems:

- Dropdown unread count examines only the 20 fetched records.
- Inbox is capped at 100 records.
- No pagination.
- No Supabase realtime subscription.
- Dropdown mutations generally ignore returned errors.
- Legacy Intake links point to /intake?id=..., which no longer opens a submission.

Email notifications

Not implemented. Only password-reset email is delegated to Supabase Auth.

12. Database and Supabase Audit

Tables

The schema defines 24 application tables.

Identity and RBAC:

- profiles
- employee_roles
- app_roles
- permissions
- role_permissions

CRM and project management:

- clients
- projects
- project_members
- tasks
- files
- interactions
- comments
- notifications

Legacy Intake:

- intake_forms
- intake_projects
- intake_attachments

Dynamic forms:

- form_templates
- form_questions
- form_submissions
- form_submission_answers
- form_submission_attachments

Portfolio:

- portfolio_categories
- portfolio_projects
- portfolio_project_images

RLS is enabled on all 24 tables.

Storage buckets

project-files

Private

MAIN POLICY

Project access plus file permissions

intake-files — Private

MAIN POLICY
Object owner

form-files — Private

MAIN POLICY
Owner or submission viewer

portfolio-images — Private

MAIN POLICY
Published image or portfolio manager

avatars — Public

MAIN POLICY
Public read; staff/owner upload

Key data-integrity concerns

1. clients.email is not unique.
2. Public client creation uses select then insert, permitting concurrent duplicates.
3. profiles.email is not unique after placeholder changes.
4. profiles.id no longer has an Auth-user FK, allowing placeholders and orphans.
5. Team deletion does not delete the Auth user.
6. comments.entity_id is polymorphic and has no foreign key.
7. Task assignees are not constrained to project members or internal users.
8. portfolio_projects.cover_image_path has no FK to an image row.
9. Storage deletion and table deletion are separate client operations, not atomic.
10. Client aggregates such as project count and total value have no maintenance trigger.
11. Form uploads can remain orphaned indefinitely.
12. Legacy Intake continues to accumulate alongside dynamic forms.

schema.sql versus migrations

They are not consistent.

- schema.sql includes portfolio, account guard, and enhanced notification definitions.
- It does not include migration 20260816000000_force_password_change_and_profile_fields.sql.
- A fresh install from schema.sql therefore lacks must_change_password, enhanced profile fields, individual social fields, other_social_links, and mark_password_changed.
- Sequential migrations add those fields.
- Neither installation path defines update_own_enhanced_profile, which the UI calls.
- The fresh-schema test does not test the profile or forced-password workflow, so it does not catch the mismatch.

Supabase configuration not verifiable

Not present in the repository:

- supabase/config.toml
- Linked project reference
- Local environment

- Anonymous Auth status
- Email signup status
- Redirect URL list
- Live bucket configuration
- Live migration history

13. Routes Audit

Public routes

/

Client landing

ACCESS AND STATUS
Public in source; deployment unavailable for verification

/auth

Login, reset, update password

ACCESS AND STATUS
Public; no signup

/forms

Published forms listing

ACCESS AND STATUS
Public; conflicts with internal Forms navigation

/f/[slug]

Dynamic public form

ACCESS AND STATUS
Public only when published

/portfolio

Public portfolio

ACCESS AND STATUS
Public source implementation

/portfolio/[slug]

Public portfolio detail

ACCESS AND STATUS
Published records only

/intake

Legacy redirect

ACCESS AND STATUS
Broken for anonymous visitors; redirects to Auth

Authenticated staff routes

/dashboard

Live overview

ACCESS AND STATUS
dashboard.view; basic

/projects

Project CRUD/list

ACCESS AND STATUS
project.view; partial

/projects/[id]

Project/tasks/progress

ACCESS AND STATUS
project.view; partial

/tasks

ACCESS AND STATUS
task.view; partial

Task board

/clients

ACCESS AND STATUS
client.view; complete basic workflow

CRM client CRUD

/clients/[id]

ACCESS AND STATUS
client.view; complete basic workflow

Client details/projects

/team

ACCESS AND STATUS
employee.view; complete

Team directory

/team/[id]

ACCESS AND STATUS
employee.view; complete

Member details

/files

ACCESS AND STATUS
file.view; needs review

Project files

/notifications

ACCESS AND STATUS
notification.view; partial

Notification inbox

/reports

ACCESS AND STATUS
report.view; partial

Basic reports

/settings

ACCESS AND STATUS
settings.view; placeholder

Link to Profile

/profile

ACCESS AND STATUS
Authenticated staff; save broken

Self profile

/templates

ACCESS AND STATUS
workspace.access; not implemented

Placeholder

/ai-assistant

Placeholder

ACCESS AND STATUS

workspace.access; not implemented

Admin routes

/admin

Tabbed administration

ACCESS AND STATUS

Requires admin.manage; partial

/admin/forms/[id]

Form builder/responses

ACCESS AND STATUS

Shell requires admin.manage, page requires form.manage

/admin/team

Direct/duplicate team managemen

ACCESS AND STATUS

Requires admin.manage

Client route

/portal

Legacy client Intake portal

ACCESS AND STATUS

Client role only; legacy

There are no Manager-only or Employee-only routes; they share permission-filtered staff pages.

14. UI and UX Findings

Strong areas

- Consistent loading and empty states
- Clear public CTAs
- Mobile public navigation
- Responsive grids
- Permission-filtered staff navigation
- Useful form success confirmation
- Clear status labels
- Good form-builder preview
- Meaningful team and portfolio empty states

Biggest UX problems

1. Internal Forms navigation opens the public website.
2. Form/portfolio permissions claim independent grantability but their UI requires full Admin access.
3. Profile forms look editable but saving fails.
4. Credentials modal promises forced password change that does not occur.
5. Admin email edits appear successful without changing login credentials.
6. Delete Team Member does not delete the login identity.
7. Builder All Forms back link returns to the default Team tab, not Forms.
8. Admin page loads unrelated datasets before the selected tab renders.
9. Task controls are shown where permissions may reject them.
10. Mobile bottom navigation may render too many items in one row.
11. Modals lack a focus trap and generally do not close with Escape.
12. Public workflows have no tracking after the success message.

15. Code Quality and Architecture Findings

Large files

- supabase/schema.sql: 3,815 lines
- lib/supabase/database.ts: 982 lines
- app/admin/forms/[id]/page.tsx: 671 lines
- app/notifications/page.tsx: 641 lines
- components/admin/team-management.tsx: 600 lines
- components/admin/portfolio-management.tsx: 455 lines
- components/admin/roles-permissions.tsx: 450 lines
- app/_components/client-landing-page.tsx: 471 lines

Architecture concerns

- Nearly all data loading is client-side.
- Supabase CRUD logic is collected in one large browser-facing database.ts.
- Admin duplicates project, client, and team operations available elsewhere.
- There is no middleware or server-side page authorization.
- Business workflows rely on client orchestration plus RLS.
- Storage upload followed by row insert/delete is not transactional.
- Manually maintained TypeScript database types have drifted from schema.
- Permission definitions exist in both the database and lib/permissions.ts.
- Marketing service lists and portfolio copy are hardcoded.
- README claims are more complete than actual implementation in several areas.

any usage

Limited, but concentrated around schema drift:

- Missing team RPC typing
- Missing enhanced-profile RPC typing
- Password-change RPC typing
- Direct profile update fallback

Dead or unused code

Likely unused components:

- components/ui/badge.tsx
- components/ui/button.tsx
- components/ui/card.tsx
- components/ui/progress.tsx
- components/ui/database-status.tsx

Unused or legacy systems:

- Interactions frontend
- Comments frontend
- Legacy Intake helpers
- AI Assistant placeholder
- Templates placeholder

- Duplicate /admin/team
- Legacy Client Portal

16. Security Audit

No confirmed unauthenticated internal-account takeover was found in the repository implementation. The Auth trigger and provisioning API are strong controls if deployed correctly.

High

IMPACT
Employees can query all submissions, answers, files, and client PII

Default Employee receives `subm

High

IMPACT
Spam can create submissions, clients, projects, notifications, and storage cost

No public-form rate limiting, C

High

IMPACT
Temporary passwords remain usable indefinitely

Forced-password enforcement is

High

IMPACT
Any authenticated caller can clear another user's flag

`mark_password_changed(p_user_

High

IMPACT
Storage abuse and orphaned objects

Form files have no MIME/size li

High

IMPACT
Displayed identity and login identity diverge

Team email changes do not updat

High

IMPACT
Account-removal expectations are violated

Team deletion does not delete A

High

IMPACT
Intended public forms and portfolio may be inaccessible

Deployment appears protected/no

Medium

IMPACT
Administrators can lock the workspace

Roles can remove the last admin

Medium

Task assignee is not constraine

IMPACT
Invalid or inaccessible assignments

Medium

Public avatar bucket lacks enfo

IMPACT
Arbitrary public content/storage abuse by authenticated users

Medium

Project file delete policies di

IMPACT
Storage may be deleted while DB row remains

Medium

No audit log

IMPACT
Sensitive changes are not attributable

Low

File metadata does not verify o

IMPACT
Fake/missing attachment records can be submitted

Low

Client-side route guards expose

IMPACT
Data remains RLS-protected, but page authorization is not server-rendered

17. Performance Audit

Dashboard fetches all accessible projects, tasks, clients, notifications

PRIORITY
High

Increasing load and browser mem

Reports fetch all projects/tasks and calculate in browser

PRIORITY
High

Poor scalability

No list pagination

PRIORITY
High

Major lists degrade as data gro

Filters are client-side

PRIORITY
Medium

Entire datasets must be downloa

Public portfolio signs every image separately

PRIORITY
High

N+1 network/database operations

Admin portfolio also hydrates signed URLs separately

PRIORITY
Medium

Slow for large galleries

Public landing/portfolio are client-rendered

PRIORITY
Medium

Slower initial content and weak

Notification dropdown polls every 20 seconds

PRIORITY
Medium

Continuous requests from every

Plain img used for portfolio/avatar images

PRIORITY
Medium

No framework image optimization

Admin preloads unrelated datasets

PRIORITY
Medium

Redundant requests before Forms

Large monolithic components

Higher maintenance and rerender

PRIORITY

Low/Medium

18. Master Implementation Checklist

Already Completed

- Email/password login
- Logout
- No public signup UI
- Anonymous dynamic-form submission
- Database-backed form definitions
- Ten form question types
- Required fields and server choice/rating/number validation
- Form lifecycle
- Form duplication and ordering
- Answer snapshots and versioning
- Submission persistence
- Admin form-submission notifications
- Public portfolio database/RLS architecture
- Portfolio publishing and project ordering
- CRM client CRUD
- Basic project CRUD
- Project membership RLS
- Project/task assignment notifications
- Private project-file download
- Team directory and filters
- Job-role catalog
- Notification persistence/read state
- Loading and empty states
- Theme/accent support

Partially Completed

- Password reset
- Team account creation
- Manager/Admin/custom-role creation
- Roles and permissions UI
- Conditional logic and sections
- Submission review
- Client journey after submission
- Portfolio gallery management
- Project management and assignments
- Tasks
- Employee, Manager, and Admin workspaces

- Search and filters
- Dashboard and reports
- Responsive/mobile behavior
- Error handling
- Client portal

Broken / Needs Immediate Fix

- Profile saving
- Forced password change
- Temporary-password enforcement
- Team email/Auth synchronization
- Team Auth-user deletion
- /intake anonymous redirect
- Internal Forms route collision
- Granular form/portfolio UI access
- schema.sql and migration mismatch
- Public deployment availability
- Form upload cleanup and limits
- Employee-wide submission/PII visibility

Not Implemented

- Business email notifications
- Global submission inbox
- Submission assignment/review workflow
- My Tasks
- Overdue task handling
- Project owner/manager/priority
- Activity/audit logs
- Workspace settings
- Templates
- AI assistant
- Pagination
- Global search
- Client tracking of dynamic submissions

Legacy / Should Be Removed or Migrated

- /intake
- intake_forms
- intake_projects
- intake_attachments
- intake-files
- Legacy Intake RPC and notification trigger
- Legacy client portal Intake-only logic
- Unused interactions/comments frontend architecture
- Duplicate /admin/team
- Placeholder Templates and AI routes if not planned

- Unused generic UI components

Recommended Improvements

- Generate temporary passwords server-side or send secure invitation links
- Add server-side route authorization/middleware
- Use generated Supabase TypeScript types
- Add aggregate/count RPCs
- Add pagination and server filtering
- Add rate limiting, CAPTCHA, and upload quotas
- Add a real submission state machine
- Add explicit project ownership
- Add realtime notifications or efficient count subscriptions
- Add CI for build, lint, database tests, and public smoke tests

19. Current Product Architecture

Client

Landing page -> Published forms list -> Dynamic form -> Anonymous Auth attempted -> Optional file uploads to form-files -> submit_dynamic_form -> Submission and answer snapshots -> CRM client match/create -> Optional unassigned project -> Admin in-app notification -> Admin response tab -> Workflow stops.

Missing after submission:

- Client confirmation email
- Client-accessible tracking
- Admin assignment queue
- Manager/employee assignment
- Contacted/reviewed workflow
- Automation beyond optional project creation

Admin

Login -> Dashboard -> Administration

- Team account creation
- Job roles
- Access roles and permissions
- Dynamic forms and responses
- Portfolio
- Legacy client accounts
- Projects
- Project assignments

-> Shared projects, tasks, files, clients, reports, notifications.

Workflow stops or breaks at profile editing, forced-password lifecycle, Auth email/delete lifecycle, submission assignment, project ownership, and business email automation.

Employee

Login -> Dashboard -> Assigned projects through RLS -> All tasks in assigned projects -> Project files -> Accessible clients -> Team directory -> Reports -> Notifications.

Missing: My Tasks, overdue work, personal workload view, dedicated submission role, task collaboration, and reliable internal Forms destination.

Manager

Login -> Dashboard -> All projects and clients -> Project/task create, edit, delete, assign -> Project files -> Submission database access -> Team directory -> Reports and notifications.

Missing: manager dashboard, submission triage, team workload, explicit manager ownership, and client communication workflow.

Legacy Client Account

Login -> /portal -> Legacy intake_forms linked to CRM client -> Request another service through /forms.

Dynamic-form submissions are not shown in this portal.

20. Proposed Implementation Roadmap

Critical Fixes

Session 1 - Reconcile database source of truth

- Objective: Make fresh schema, migrations, and TypeScript types represent one database.
- Current problem: schema.sql omits migration 20260816 and required profile RPC is absent.
- Implement: Rebuild authoritative schema; add enhanced profile RPC; regenerate types; document order.
- Dependencies: None.
- Complexity: Medium.
- Expected result: Fresh and upgraded databases expose identical columns/functions.
- Testing: Fresh-schema PGlite, sequential migrations, schema/type comparison.

Session 2 - Repair profile editing

- Objective: Make every visible profile field persist safely.
- Current problem: Save calls a nonexistent RPC and can orphan avatar uploads.
- Implement: Secure self-update RPC; field allowlist; update UI; clean failed uploads.
- Dependencies: Session 1.
- Complexity: Medium.
- Expected result: Users edit permitted fields without changing role/status/email.
- Testing: Employee/Admin saves, forbidden-field RPC tests, avatar cleanup.

Session 3 - Implement temporary-password enforcement

- Objective: Require new users to replace temporary credentials.
- Current problem: Flag never set and redirect is broken/bypassable.
- Implement: Set flag at provisioning; self-only clear; dedicated route; backend restrictions.
- Dependencies: Sessions 1 and 2.
- Complexity: Medium/High.
- Expected result: Temporary credentials cannot access normal workspace operations.
- Testing: First login, direct URLs, direct Supabase calls, reset flow, second login.

Session 4 - Complete team Auth lifecycle

- Objective: Synchronize profile and Supabase Auth identities.
- Current problem: Email edits/deletion affect only profiles.
- Implement: Protected update/delete endpoints; Auth email update; Auth-user deletion; rollback; reset/invite.
- Dependencies: Session 3.
- Complexity: High.
- Expected result: Create/edit/deactivate/delete affect the complete account.
- Testing: Login after email change, deletion revocation, last-admin protection, rollback.

Session 5 - Fix authorization matrix and PII exposure

- Objective: Align routes, permissions, and RLS.
- Current problem: Employees can query all submissions; granular tools cannot reach UI.
- Implement: Revise default matrix; split admin routes; preserve last-admin invariant.
- Dependencies: Session 1.
- Complexity: High.
- Expected result: Every checkbox corresponds to intended UI and backend access.
- Testing: Default/custom role matrix, direct PostgREST attempts, lockout tests.

Session 6 - Restore public deployment access

- Objective: Make landing, forms, and portfolio publicly reachable.
- Current problem: Discoverable URLs are protected or 404.
- Implement: Correct Vercel alias/protection, homepage, environment, and redirect URLs.
- Dependencies: None; coordinate Auth redirects with Session 3.
- Complexity: Low/Medium.
- Expected result: Incognito visitors can access public routes.
- Testing: External smoke tests for /, /forms, /f/[slug], /portfolio, /auth, /intake.

Core Workflow

Session 7 - Harden public form submission

- Objective: Protect forms and storage from abuse.
- Current problem: Unlimited submissions, payloads, and uploads.
- Implement: Rate limit, CAPTCHA/honeypot, payload limits, bucket limits, MIME allowlist.
- Dependencies: Session 1.
- Complexity: High.
- Expected result: Public forms remain usable without becoming a spam/storage vector.
- Testing: Rate limits, oversized payloads, invalid MIME, anonymous submit, accessibility.

Session 8 - Complete submission triage

- Objective: Turn a response into an actionable Admin workflow.
- Current problem: Responses support only submitted/archived.
- Implement: Global inbox; reviewed/contacted/rejected; assignee; notes; filters; pagination; project linking.
- Dependencies: Sessions 5 and 7.
- Complexity: High.
- Expected result: Admin processes every request without searching individual builders.
- Testing: State transitions, permissions, assignment notifications, filters, history.

Session 9 - Retire legacy Intake

- Objective: Establish one form architecture.
- Current problem: Legacy tables, RPC, storage, and portal remain active.
- Implement: Migrate data; server redirect /intake; move portal reads; retire legacy writes.
- Dependencies: Session 8.
- Complexity: High.
- Expected result: Form Builder -> dynamic forms -> dynamic submissions is the only system.
- Testing: Old links, historical responses, notification links, no legacy writes.

Session 10 - Add project ownership and assignment workflow

- Objective: Ensure projects have accountable owners.
- Current problem: No owner/manager/priority; auto-created projects unassigned.
- Implement: Owner/manager/priority or project-role model; triage assignment; constraints.
- Dependencies: Session 8.
- Complexity: Medium/High.
- Expected result: Every active project has a responsible manager/team.
- Testing: RLS, auto-created assignment, reassignment, inactive users.

Session 11 - Complete employee task workflow

- Objective: Provide a usable employee workspace.
- Current problem: No My Tasks/overdue; assignee selector incomplete.

- Implement: My Tasks, overdue/due-soon, project-member assignees, task detail, permission-aware controls.
- Dependencies: Session 10.
- Complexity: Medium.
- Expected result: Employees understand exactly what they should do next.
- Testing: Employee/manager views, overdue calculations, assignment integrity, direct API tests.

Client Experience

Session 12 - Dynamic client request tracking

- Objective: Let clients track modern submissions without becoming staff.
- Current problem: Submitters receive only a success screen; portal reads Intake only.
- Implement: Secure status token or client linkage; confirmation page; privacy model.
- Dependencies: Sessions 8 and 9.
- Complexity: High.
- Expected result: Clients can check request status without internal access.
- Testing: Token isolation, legacy migration, revoked/expired links.

Session 13 - Portfolio gallery completion

- Objective: Finish portfolio media management.
- Current problem: Gallery order and alt text are not editable.
- Implement: Image reorder RPC/UI, alt-text editor, batch progress, failure recovery.
- Dependencies: Session 1.
- Complexity: Medium.
- Expected result: Admin controls the exact public gallery.
- Testing: Public order, draft protection, upload cleanup, mobile gallery.

Automation

Session 14 - Notification completeness

- Objective: Notify users for actionable workflow changes.
- Current problem: No task status/due reminders, submission assignment, or unassignment events.
- Implement: Event definitions, completion/due/overdue notifications, assignment notifications, preferences.
- Dependencies: Sessions 8 and 11.
- Complexity: Medium.
- Expected result: Notifications correspond to actionable events.
- Testing: Trigger idempotency, recipient correctness, links, RLS.

Session 15 - Email notification service

- Objective: Add business email notifications.
- Current problem: Only Supabase password-reset email exists.
- Implement: Provider, server/Edge sender, templates, retries, logs, opt-outs.
- Dependencies: Session 14.
- Complexity: High.
- Expected result: Admins and assignees receive reliable configured emails.
- Testing: Provider sandbox, retries, duplicate prevention, secret safety, unsubscribe.

Admin Improvements

Session 16 - Admin information architecture

- Objective: Replace oversized Admin tabs with capability routes.
- Current problem: Monolithic page, redundant loading, confusing navigation.

- Implement: Separate team, roles, forms, portfolio, submissions, and assignments routes.
- Dependencies: Session 5.
- Complexity: Medium.
- Expected result: Users land directly in permitted tools.
- Testing: Route matrix, back navigation, deep links, mobile navigation.

Session 17 - Reports and audit history

- Objective: Add operational visibility and accountability.
- Current problem: Reports are basic and no audit log exists.
- Implement: Audit events, activity feed, date filters, workload/submission metrics.
- Dependencies: Sessions 8 to 11.
- Complexity: High.
- Expected result: Admins can answer who changed what and measure work.
- Testing: Immutability, redaction, report accuracy, RLS.

UX, Performance, and Cleanup

Session 18 - Pagination and server-side queries

- Objective: Make lists, dashboards, and reports scale.
- Current problem: Full-table downloads and browser-side counting/filtering.
- Implement: Pagination, aggregate RPCs, server filters, efficient unread counts.
- Dependencies: Stable earlier schemas.
- Complexity: Medium/High.
- Expected result: Predictable query cost as data grows.
- Testing: Large dataset, query counts, boundaries, RLS.

Session 19 - Public rendering and image performance

- Objective: Improve public first load and portfolio efficiency.
- Current problem: Client-only public data and per-image signed URL requests.
- Implement: Server rendering, batched URLs, optimized/lazy images, caching.
- Dependencies: Session 6.
- Complexity: Medium.
- Expected result: Faster landing/portfolio and better SEO.
- Testing: Lighthouse, cache visibility, draft leakage, expired URLs.

Session 20 - Cleanup and CI

- Objective: Remove dead code and expose regressions early.
- Current problem: Legacy helpers, unused components, placeholders, no automated test check.
- Implement: Remove approved legacy/dead code; add build/lint/PGlite CI; smoke tests; generated types.
- Dependencies: Sessions 1 and 9; product decision on placeholders.
- Complexity: Medium.
- Expected result: Smaller codebase and dependable release gates.
- Testing: Full build/lint/database suite, route smoke tests, clean import analysis.

Final Assessment

What has already been built?

A strong Supabase-backed operations foundation: closed authentication, granular RBAC/RLS, dynamic forms, a public portfolio, CRM clients, projects, tasks, files, team management, and persistent notifications.

What is working?

The core dynamic-form database workflow, public portfolio architecture, basic CRM/project/task workflows, role-based database enforcement, team directory, notification persistence, and the public landing source implementation.

What is broken?

Profile saving, forced temporary-password change, complete team Auth lifecycle, anonymous /intake redirect, granular admin-tool routing, schema consistency, and likely public deployment access.

What still needs to be built?

Submission triage, anti-abuse controls, client tracking, My Tasks/overdue workflows, project ownership, email notifications, audit logs, settings, pagination, scalable reporting, and retirement of the legacy Intake system.